



Amazon CloudFront

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

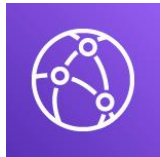
Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

Introduction to CloudFront

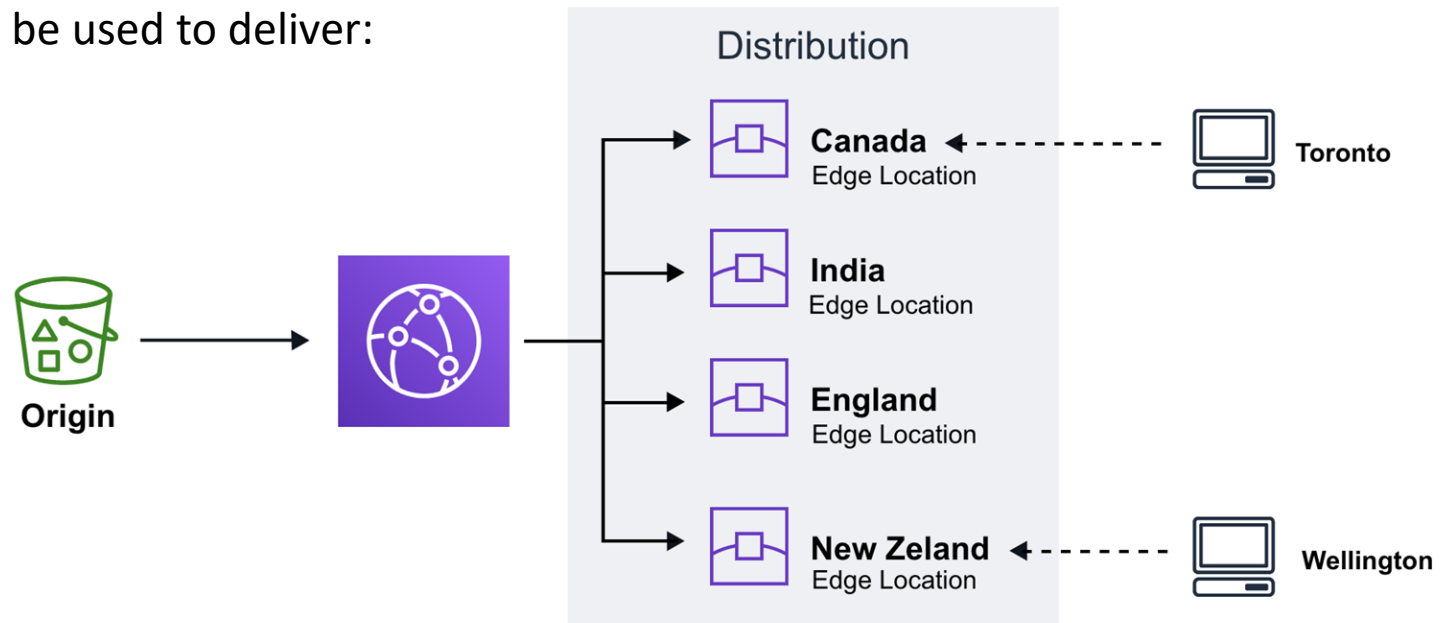
Content Delivery Network (CDN)

A CDN is a distributed network of servers that delivers web pages and content to users based on their **geographical location**, the **origin of the webpage**, and a **content delivery server**.



CloudFront is a CDN that can be used to deliver:

- Static content
- Dynamic content
- Streaming Videos
- Web Sockets

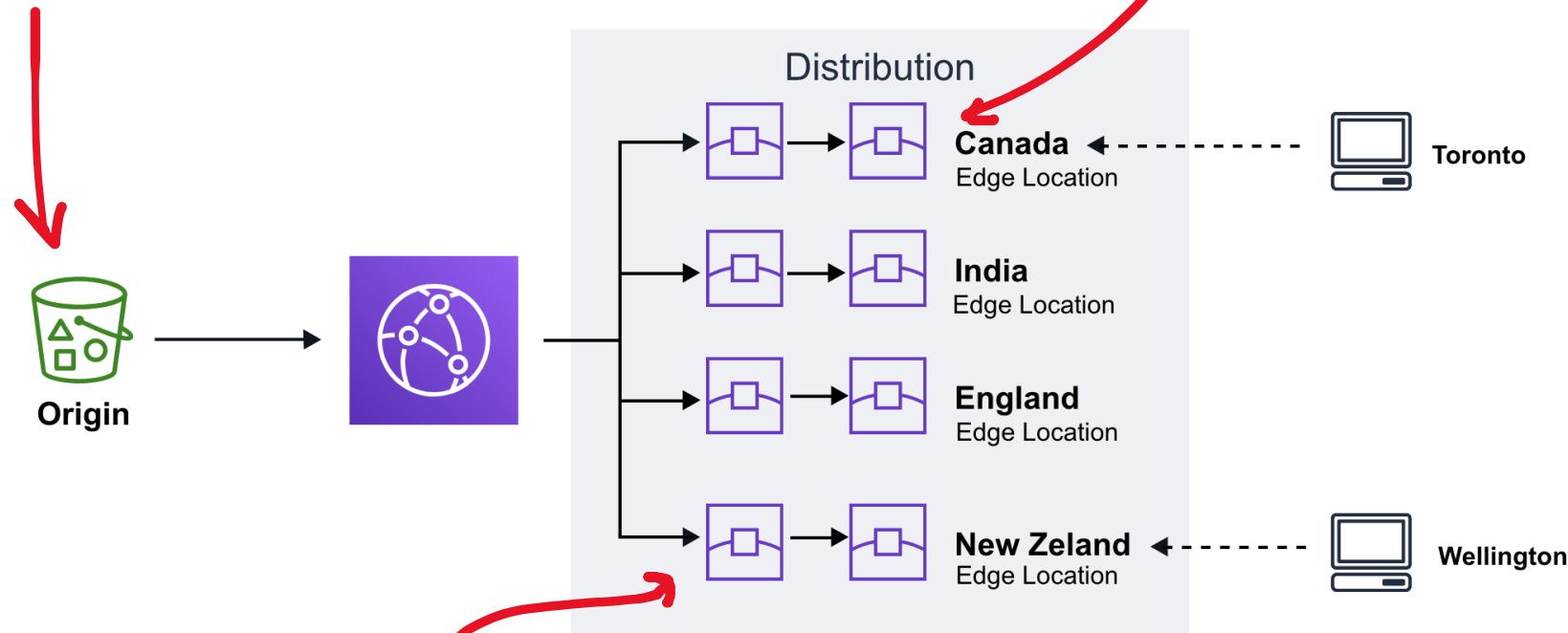


- Amazon CloudFront can be fronted with AWS WAF for OSWAP TOP 10 protection
- Amazon CloudFront can stream Videos On Demand using ISS Microsoft Smooth Streaming

CloudFront - Core Components

Origin is location where all of original files are located. For example an S3 Bucket, EC2 Instance, ELB, or Route53

Edge Location is compute located strategically close to end users

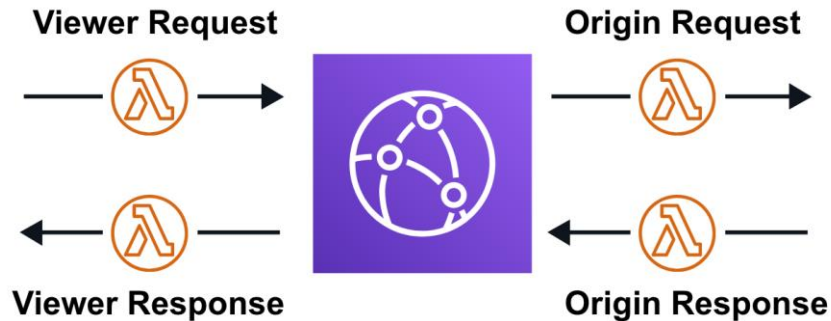


Regional Caches is compute located in broad geographic locations to speed up requests for edge locations

Distribution A collection of Edge locations and Regional Cache that defines how cached content should behave

CloudFront – Lambda@Edge

Lambda@Edge are lambda functions to **override the behaviour** of requests and responses



Functions are deployed at **Regional Edge Caches**

There are 4 functions for Lambda@Edge

1. **Viewer request** — When CloudFront receives a request from a viewer
2. **Origin request** — Before CloudFront forwards a request to the origin
3. **Origin response** — When CloudFront receives a response from the origin
4. **Viewer response** — Before CloudFront returns the response to the viewer

Supported languages:

- Python
- Node.js

Viewer Request Use-Cases

- Redirect HTTP to HTTPS.
- Inspect cookies for user authentication.
- Modify headers for A/B testing.

Viewer Response Use-Cases

- Add security headers (e.g., CSP, HSTS).
- Set cookies for client-side tracking.
- Customize error messages.

Origin Request Use-Cases

- Rewrite URLs for SEO or routing.
- Inject headers for origin authentication.
- Selective content serving based on user-agent.

Origin Response Use-Cases

- Modify headers to control caching.
- Update URLs in HTML for versioning.
- Customize error responses from the origin.

CloudFront - Lambda@Edge

Viewer Request Function example: **Redirecting HTTP to HTTPS**



```
def lambda_handler(event, context):
    request = event['Records'][0]['cf']['request']
    # Perform operations on the request. Example: Redirect HTTP to HTTPS
    if request['headers']['cloudfront-forwarded-proto'][0]['value'] != 'https':
        return {
            'status': '301',
            'statusDescription': 'Moved Permanently',
            'headers': {
                'location': [{
                    'key': 'Location',
                    'value': 'https://' + request['headers']['host'][0]['value'] + request['uri']
                }]
            }
        }
    # Return the original request for CloudFront to process
    return request
```

CloudFront - Lambda@Edge

Viewer Response Function example: **Add Security Headers**



```
def lambda_handler(event, context):
    response = event['Records'][0]['cf']['response']
    # Add security headers to the response
    headers = response['headers']
    headers['x-content-type-options'] = [{'key': 'X-Content-Type-Options', 'value': 'nosniff'}]
    headers['x-frame-options'] = [{'key': 'X-Frame-Options', 'value': 'DENY'}]
    headers['x-xss-protection'] = [{'key': 'X-XSS-Protection', 'value': '1; mode=block'}]
    # Return the modified response
    return response
```

CloudFront - Lambda@Edge

Origin Request Function example: **Serve different version based on User Agent**



```
def lambda_handler(event, context):
    request = event['Records'][0]['cf']['request']
    # Modify the request URI to serve a different version based on user-agent
    user_agent = request['headers'].get('user-agent', [{'value': ''}])[0]['value']
    if 'mobile' in user_agent.lower():
        request['uri'] = '/mobile' + request['uri']
    else:
        request['uri'] = '/desktop' + request['uri']
    # Return the modified request
    return request
```

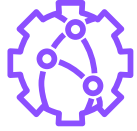
CloudFront - Lambda@Edge

Origin Response Function example: **Change 404 to 200 and provide custom response body**

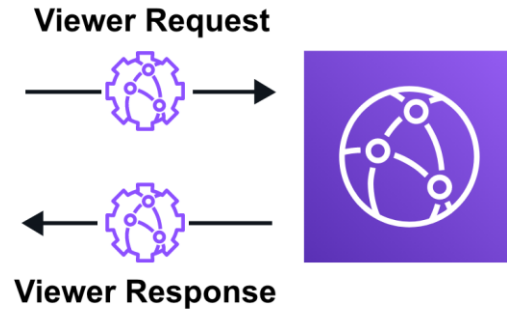


```
def lambda_handler(event, context):
    response = event['Records'][0]['cf']['response']
    # Check for 404 status code and modify the response
    if response['status'] == '404':
        response['status'] = '200'
        response['statusDescription'] = 'OK'
        response['body'] = 'Custom error page content'
        response['headers']['content-type'] = [{'key': 'Content-Type', 'value': 'text/html'}]
    # Return the modified response
    return response
```

CloudFront Functions



CloudFront Functions are **lightweight edge functions** for high-scale, latency-sensitive CDN customizations. CloudFront Functions are **cheaper, faster, but more limited than Lambda@Edge functions**



Functions are deployed at **Edge Locations**

There are 2 functions for CloudFront Functions

1. **Viewer request** — When CloudFront receives a request from a viewer
2. **Viewer response** — Before CloudFront returns the response to the viewer

Supported languages:

- JavaScript (ECMAScript 5.1 compliant)

Use-cases:

- Cache key normalization
- Header manipulation
- Status code modification and body generation
- URL redirects or rewrites
- Request authorization

CloudFront Functions

Viewer Request Example Function: **Redirect HTTP to HTTPS**



```
function handler(event) {
  var request = event.request;
  var headers = request.headers;
  // Check if the viewer request is using HTTP
  if (request.uri.startsWith('http://')) {
    // Generate an HTTP redirect response to HTTPS
    var response = {
      statusCode: 301,
      statusDescription: 'Moved Permanently',
      headers: {
        "location": {
          value: "https://" + headers.host.value + request.uri
        }
      }
    };
    return response;
  }
  return request;
}
```

CloudFront Functions

Viewer Response Function example: **Add Security Headers**



```
function handler(event) {  
  var response = event.response;  
  var headers = response.headers;  
  // Set HTTP security headers  
  headers['x-content-type-options'] = { value: 'nosniff'};  
  headers['x-frame-options'] = { value: 'DENY'};  
  headers['x-xss-protection'] = { value: '1; mode=block'};  
  headers['referrer-policy'] = { value: 'same-origin'};  
  // Return the modified response  
  return response;  
}
```

CloudFront Functions vs Lambda@Edge

	CloudFront Functions	Lambda@Edge
Programming languages	JavaScript (ECMAScript 5.1 compliant)	Node.js and Python
Event sources	Viewer request/response	- Viewer request/ response - Origin request/ response
Scale	10,000,000 requests per second or more	Up to 10,000 requests per second per Region
Function duration	Submillisecond	- Up to 5 seconds (viewer request and response) - Up to 30 seconds (origin request and origin response)
Maximum memory	2 MB	128-3,008 MB
Maximum size function code + libs	10 KB	1 MB (viewer request and viewer response) 50 MB (origin request and origin response)
Network, File System, and Request Body access	No	Yes
Access to geolocation and device data	Yes	- No (viewer request) - Yes (origin request/response, and viewer response)
Can build/test entirely within CloudFront		No
Function logging and metrics		Yes
Pricing	Free tier available; charged per request	No free tier; charged per request, function duration

CloudFront –Origin

CloudFront Origin is the source **where CloudFront will send requests.**

Domain Name: the address to the origin

Origin Path: the path at the specified address

Configurations for Origins:

- **S3OriginConfig**
 - Amazon S3
- **CustomOriginConfig**
 - AWS Elemental MediaStore Container
 - Application Load Balancer
 - Lambda function URL
 - HTTP Server eg. Amazon EC2 or another custom origin)
 - CloudFront Origins Group

```
"Origins": {
  "Quantity": 1,
  "Items": [
    {
      "Id": "awsexamplebucket.s3.amazonaws.com-cli-example",
      "DomainName": "awsexamplebucket.s3.amazonaws.com",
      "OriginPath": "",
      "CustomHeaders": {
        "Quantity": 0
      },
      "S3OriginConfig": {
        "OriginAccessIdentity": ""
      }
    }
  ]
},
```

You provide this configuration when creating the distribution

CloudFront – Origin Groups

Origin Groups allows you to **group a primary and secondary origin together as a destination for requests** enabling **high availability and fault tolerance**

An origin includes two origins:

- Primary Origin — where requests will be send.
- Secondary Origin — a **failover origin** that will receive traffic over the primary origin if the failover criteria is met

The Failover Criteria is based chosen HTTP Status Codes:

- 400, 403, 404, 416, 500, 502, 503, or 504

CloudFront will only fail over on GET, HEAD and OPTIONS Requests

- It will not failover on PUT, POST, PATCH and DELETE

You can use different custom error pages for each origin.

```
"OriginGroups": {
  "Quantity": 1,
  "Items": [
    {
      "Id": "OriginGroup1",
      "FailoverCriteria": {
        "StatusCodes": {
          "Quantity": 1,
          "Items": [500]
        }
      },
      "Members": {
        "Quantity": 2,
        "Items": [
          {"OriginId": "OriginId1"},
          {"OriginId": "OriginId2"}
        ]
      }
    }
  ]
}
```



CloudFront – Caching Policy

The Cache Key is the unique identifier for every object in the cache and it determines whether a viewer request results in a **cache hit**.

A cache hit occurs when a viewer request generates the same cache key as a prior request, and the object for that cache key is in the edge location's cache.

A higher cache hit ratio (more people hitting the cache) will result in better performance.

A **Cache Policy** is used to control the cache key.

AWS has managed Cache Policies for use:

- Amplify
- CachingDisabled
- CachingOptimized
- **CachingOptimizedForUncompressedObjects**
- Elemental-MediaPackage

```
aws cloudfront update-distribution \  
--id EDFDVBD6EXAMPLE \  
--distribution-config file://modified-config.json
```

```
"DefaultCacheBehavior": {  
  "CachePolicyId": "658327ea-f89d-4fab-a63d-7e88639e58f6"  
  ...  
}
```

Settings for CachingOptimizedForUncompressedObjects:

- **Minimum TTL:** 1 second
- **Maximum TTL:** 31,536,000 seconds (365 days)
- **Default TTL:** 86,400 seconds (24 hours)
- **Headers included in the cache key:** None
- **Cookies included in the cache key:** None
- **Query strings included in the cache key:** None
- **Cache compressed objects setting:** Disabled

CloudFront – Caching Policy

You can **create a custom cache policy**.



TTL Settings:

- Minimum TTL — minimum time to hold a file in the cache
- Maximum TTL — maximum time to hold a file in the cache
- Default TTL — default time to hold a file in the cache

If all values are 0 then CloudFront caching is disabled

Cache Key Settings:

- Headers — which headers are part of the cache key
- Cookies — which cookies are part of the cache key
- Query String — which query string params are part of the cache key
- Compression Support — what compression format is part of the cache key

```
{
  "Name": "MyCustomCachePolicy",
  "Comment": "Example cache policy for specific content",
  "DefaultTTL": 86400,
  "MaxTTL": 31536000,
  "MinTTL": 60,
  "ParametersInCacheKeyAndForwardedToOrigin": {
    "EnableAcceptEncodingGzip": true,
    "EnableAcceptEncodingBrotli": false,
    "HeadersConfig": {
      "HeaderBehavior": "whitelist",
      "Headers": {
        "Quantity": 2,
        "Items": [
          "Origin",
          "Access-Control-Request-Headers"
        ]
      }
    },
    "CookiesConfig": {"CookieBehavior": "none"},
    "QueryStringConfig": {"QueryStringBehavior": "all"}
  }
}
```

CloudFront – Origin Request Policy

Origin Request is when there is a *cache miss*, and CloudFront must reach out to Origin to retrieve data.

Origin request always returns the following:

- URL Path
- Request Body (if present)
- Specific HTTP Headers
 - Host
 - User Agent
 - X-Amz-Cf-Id

Origin request doesn't include these by default:

- URL Query String
- Cookies
- All other HTTP Headers
 - *Legacy caching always forwarded headers*

Cache Key information (set by the cache policy) are automatically included in origin requests

- URL query strings
- HTTP headers
- Cookies

Origin Request Policy to retrieve additional information at the Origin about a request and also maintain a good cache hit ratio

Use the Origin Request Policy to include information you want in the request but not in the Cache Key.

AWS has the following Managed Origin Request Policies

- AllViewer
- AllViewerAndCloudFrontHeaders-2022-06
- AllViewerExceptHostHeader
- CORS-CustomOrigin
- CORS-S3Origin
- Elemental-MediaTailor-PersonalizedManifests
- UserAgentRefererHeaders

CloudFront – Origin Request Policy

Create your own policy



```
{
  "Comment": "Complex Origin Request
Policy for Personalized Content",
  "Name": "PersonalizedContentPolicy",
  "HeadersConfig": {
    "HeaderBehavior": "whitelist",
    "Headers": {
      "Quantity": 4,
      "Items": [
        "Accept-Language",
        "User-Agent",
        "Authorization",
        "X-Custom-Header"
      ]
    }
  },
  "CookiesConfig": {
    "CookieBehavior": "whitelist",
    "Cookies": {
      "Quantity": 3,
      "Items": [
        "session-id",
        "user-preferences",
        "login-token"
      ]
    }
  },
  "QueryStringConfig": {
    "QueryStringBehavior": "whitelist",
    "QueryStrings": {
      "Quantity": 2,
      "Items": ["page", "sort"]
    }
  }
}
```

Head Behavior:

- **none:** Excludes all viewer request headers except those specified in a Cache Policy.
- **whitelist:** Includes only specified headers in the Headers list.
- **allViewer:** Includes all headers from viewer requests.
- **allViewerAndWhitelistCloudFront:** Includes all viewer request headers and additional specified CloudFront headers.
- **allExcept:** Includes all viewer request headers except those specified in the Headers list.

Cookie Behavior:

- **none:** Excludes all viewer request cookies, except those in a Cache Policy.
- **whitelist:** Includes only specified cookies in CookieNames.
- **all:** Includes all viewer request cookies.
- **allExcept:** Includes all cookies except those in CookieNames.

Query String Behavior:

- **none:** Excludes all viewer request query strings, except those in a Cache Policy.
- **whitelist:** Includes only specified query strings in QueryStringNames.
- **all:** Includes all viewer request query strings.
- **allExcept:** Includes all query strings except those in QueryStringNames.

CloudFront – Response Header Policy

Response Header Policy allows you to modify the HTTP Headers in the response that viewers will see.

Headers which cannot be removed with a policy:

- Connection
- Content-Encoding
- Content-Length
- Expect
- Host
- Keep-Alive
- Proxy-Authenticate
- Proxy-Authorization
- Proxy-Connection
- Trailer
- Transfer-Encoding
- Upgrade
- Via
- Warning
- X-Cache
- X-Edge-.*
- X-Forwarded-
- X-Amz-Cf-.*
- X-Amzn-Auth
- X-Amzn-Cf-Billing
- X-Amzn-Cf-Id
- X-Amzn-Cf-Xff
- X-Amzn-ErrorType
- X-Amzn-Fle-Profile
- X-Amzn-Header-Count
- X-Amzn-Header-Order
- X-Amzn-Lambda-Integration-Tag
- X-Amzn-RequestId
- X-Accel-Buffering
- X-Accel-Charset
- X-Accel-Limit-Rate
- X-Accel-Redirect

AWS has the following managed Response Header Policies

- CORS-and-SecurityHeadersPolicy
- CORS-With-Preflight
- CORS-with-preflight-and-SecurityHeadersPolicy
- SecurityHeadersPolicy
- SimpleCORS

CloudFront – Response Header Policy

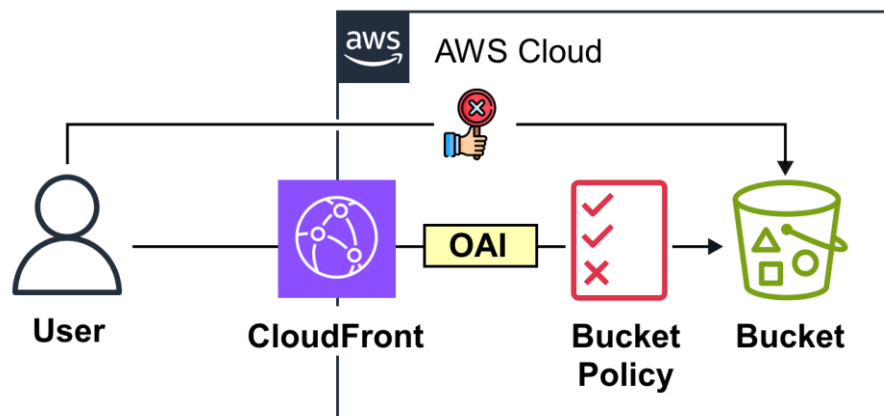
Example Custom Response Header Policy:

- Adds a Content Security Policy header to restrict resources the browser loads for a page.
- Enables the X-Content-Type-Options header with nosniff.
- Sets X-Frame-Options to DENY to prevent clickjacking attacks.
- Specifies a Referrer Policy of no-referrer.
- Configures Strict-Transport-Security to enforce HTTPS for two years, including subdomains, and requests preloading in browsers.
- Enables XSS Protection with block mode and specifies a reporting URI.
- Adds a custom header X-Custom-Header with the value CustomValue.

```
{
  "Comment": "My security headers policy",
  "Name": "SecurityHeadersPolicy",
  "SecurityHeadersConfig": {
    "ContentSecurityPolicy": {
      "Override": true,
      "ContentSecurityPolicy": "default-src 'self'; img-src 'self' https://example.com;"
    },
    "ContentTypeOptions": { "Override": true },
    "FrameOptions": { "Override": true, "FrameOption": "DENY" },
    "ReferrerPolicy": { "Override": true, "ReferrerPolicy": "no-referrer" },
    "StrictTransportSecurity": {
      "Override": true,
      "IncludeSubdomains": true,
      "Preload": true,
      "AccessControlMaxAgeSec": 63072000
    },
    "XSSProtection": {
      "Override": true,
      "ModeBlock": true,
      "Protection": true,
      "ReportUri": "https://example.com/report-xss"
    }
  },
  "CustomHeadersConfig": {
    "Items": [
      {
        "Header": "X-Custom-Header",
        "Value": "CustomValue",
        "Override": true
      }
    ]
  }
}
```

CloudFront – Origin Access Identity

Origin access identity (OAI) is a **legacy method** to ensure that an S3 Bucket is only accessed through CloudFront and not directly access via the Internet



```
{
  "Sid": "OAI",
  "Effect": "Allow",
  "Principal": {
    "AWS": "arn:aws:iam::cloudfront:user/CloudFront Origin Access Identity <origin access identity ID>"
  },
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::<S3 bucket name>/*"
}
```

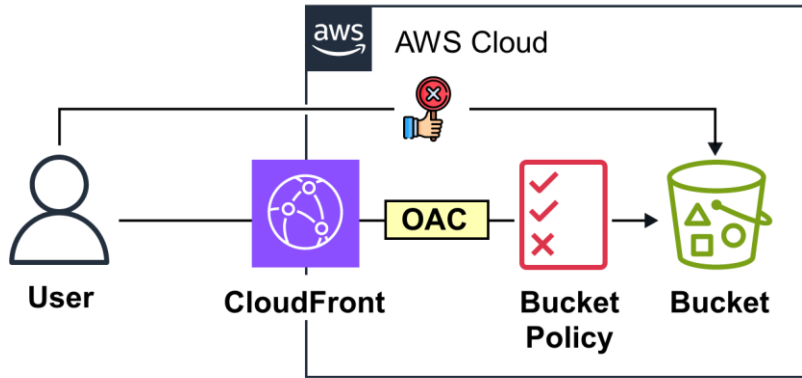
OAI worked by create a **CloudFront User** that would be then granted access via the S3 Bucket Policy

Limitations:

- not supporting granular policy configurations
- Does not sign HTTP and HTTPS requests that use the POST method in AWS regions that require AWS Signature Version 4 (SigV4)
- Does not integrate with SSE-KMS
- only be supported in existing AWS regions and regions before December 2022

CloudFront – Origin Access Control

Origin access control (OAC) to ensure that an **S3 Bucket is always access through CloudFront** and not directly access via the Internet



- Enhanced security practices
 - short term credentials
 - resource-based policies
 - frequent credential rotations
- Protects against confused deputy problem attacks
 - where an entity that doesn't have permission to perform an action can coerce a more-privileged entity to perform the action.
- Comprehensive HTTP methods support
 - GET, PUT, POST, PATCH, DELETE, OPTIONS, and HEAD.
- Supports downloading and uploading S3 objects encrypted with SSE-KMS.
- supports accessing S3 in all AWS regions

```
{
  "Sid": "OAC",
  "Effect": "Allow",
  "Principal": {
    "Service": "cloudfront.amazonaws.com"
  },
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::<S3 bucket name>/*",
  "Condition": {
    "StringEquals": {
      "AWS:SourceArn": "arn:aws:cloudfront::123456789012:distribution/<CloudFront Dist ID>"
    }
  }
}
```

Uses **IAM service principal** to authenticate with S3 origins via the S3 Bucket Policy

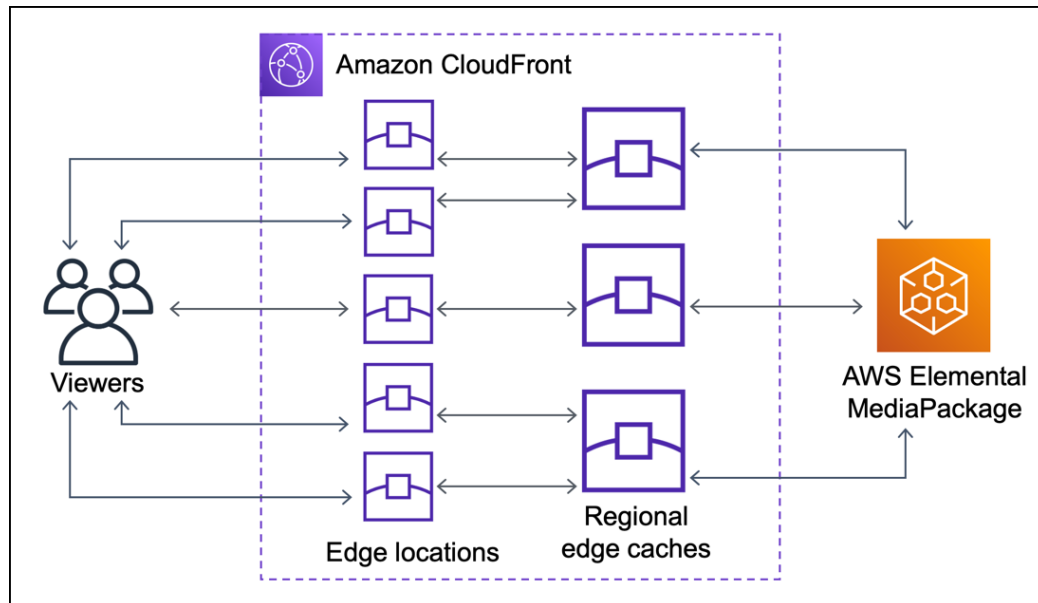
CloudFront – Origin Shield

CloudFront Origin Shield is an additional layer in the CloudFront caching infrastructure that helps to minimize your origin's load, improve its availability, and reduce its operating costs.

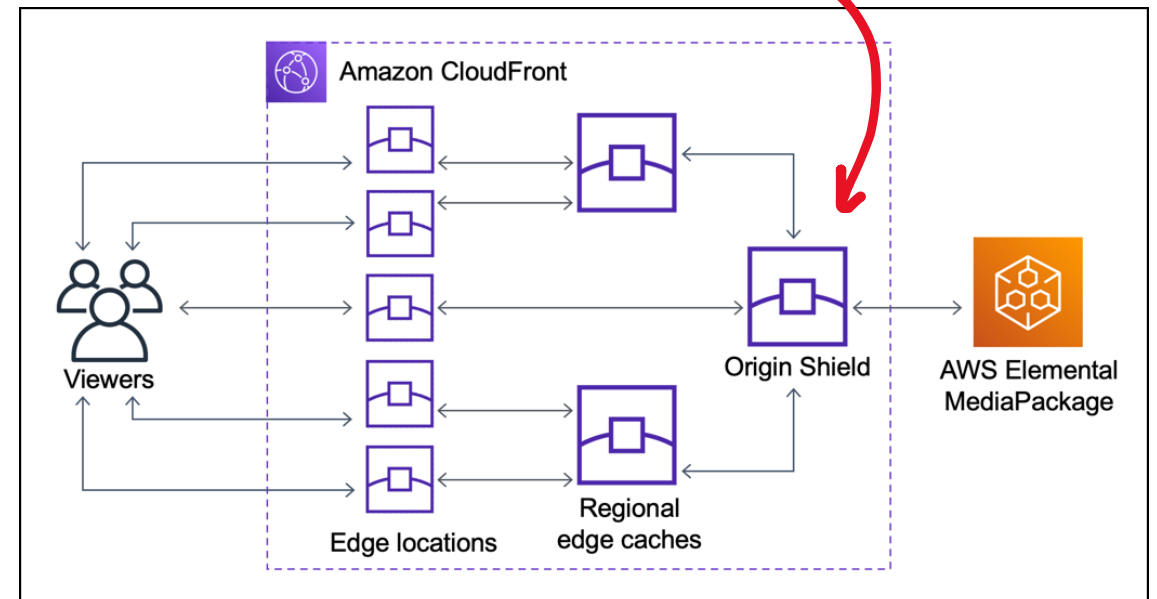
Following benefits:

- Better cache hit ratio
- Reduced origin load
- Better network performance

Without Origin Shield



With Origin Shield



CloudFront – Geographic Restrictions

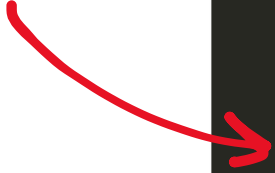
Geographic Restrictions prevents users from specific geographic locations from accessing your content served by CloudFront.



CloudFront returns an HTTP status code 403 (Forbidden) to the user.

- Geographic restrictions apply to an entire distribution.
- You **provide the country code** to which you want to allow or deny.

```
{
  "DistributionConfig": {
    ...
    "Restrictions": {
      "GeoRestriction": {
        "RestrictionType": "whitelist"
        "Quantity": 2,
        "Items": ["US", "CA"]
      }
    },
    ...
  }
}
```

A red curved arrow originates from the text 'provide the country code' in the list above and points to the 'Items' array in the JSON code, specifically highlighting the country codes 'US' and 'CA'.

CloudFront – Caching Query String Parameters

What is a query string?

A query string is a part of a URL that assigns values to **specified parameters** that can then be parsed by the web-application or browser client.

`http://www.example.com/ ?fruit=apple&count=5`

- CloudFront can forward query string to an Origin. (Origin Request Policy)
- CloudFront cache key can be based off query parameters (Cache Policy)

The reason you might want to cache key to factor in query string could be to **cache content based on different languages**.

- `https://d111111abcdef8.cloudfront.net/main.html?language=de`
- `https://d111111abcdef8.cloudfront.net/main.html?language=en`
- `https://d111111abcdef8.cloudfront.net/main.html?language=es`
- `https://d111111abcdef8.cloudfront.net/main.html?language=fr`
- `https://d111111abcdef8.cloudfront.net/main.html?language=jp`

CloudFront **does not by default cache or forward** query strings

CloudFront – Caching Cookie-based Content

What is a Cookie?

A cookie for a web browser is **key/pair data sent from a website and stored on the user's computer** by the user's web browser while the user is browsing.

```
document.cookie = "first_name=Andrew; last_name=brown";
```

Example of setting a cookie in a web-browser client js console

The **Set-Cookie** Header will contain data and other things about the cooking

```
Set-Cookie first_name=Andrew; last_name=brown; Expires=Wed, 09 Jun 2050 10:18:14 GMT; Path=/; Domain=example.com; Secure; HttpOnly
```

- CloudFront can forward cookies to an Origin. (Origin Request Policy)
- CloudFront cache key can be based cookies (Cache Policy)

You could use the Cookies values to cache different pages, you could have a language=EN key pair value in the cookie to server content for different languages

CloudFront **does not by default cache or forward** cookies

CloudFront – Content Types

CloudFront servers **don't determine the MIME type** for the objects that they serve.

What is a MIME/Media Type?

A media type (also known as a Multipurpose Internet Mail Extensions or MIME type) indicates the nature and format of a document, file, or assortment of bytes.

Web browsers use MIME type header and not the file extension to determine how to process a URL of a file. If the MIME type is not correctly set then the files may not work as expected.

Example of MIME types:

- text/plain: Plain text
- text/html: HTML documents
- text/css: Cascading Style Sheets
- text/javascript: JavaScript files
- image/jpeg: JPEG images
- image/png: PNG images
- image/gif: GIF images
- image/svg+xml: SVG (Scalable Vector Graphics) images
- application/json: JSON format
- application/xml: XML documents
- application/pdf: PDF documents

When you upload a file to your origin, we recommend that you set the **Content-Type header** field for it.

```
aws s3 cp index.html s3://my-website-bucket/index.html \
--content-type "text/html"
```

Setting Content-Type when uploading to S3

CloudFront – Invalidating Cache

When you need to expire all or specific files in the cache before the TTL you can use the **Create-Invalidation** API Action
Its best to **invalidate only what you need** because invalidating the entire cache can be expensive.

Invalidate **all files** in the cache

```
aws cloudfront create-invalidation \  
--distribution-id EDFDVBD6EXAMPLE \  
--paths "/*"
```

Invalidate **specific files** in the cache

```
aws cloudfront create-invalidation \  
--distribution-id EDFDVBD6EXAMPLE \  
--paths "/index.html" "/images/logo.png"
```

Invalidate a **batch of files** in the cache

```
aws cloudfront create-invalidation \  
--distribution-id EDFDVBD6EXAMPLE \  
--invalidation-batch file://invalidation-paths.json
```

```
{  
  "Paths": {  
    "Quantity": 2,  
    "Items": [  
      "/path/to/object",  
      "/another/path/to/object*"  
    ]  
  },  
  "CallerReference": "my-request-1"  
}
```

CloudFront – Cache Busting with Digests

What is a Cache Digest?

A cache digest is a value that appended to the end of a filename with the intention to *bust a cache*.
Cache busting is a way to prevent browsers from caching your content.

```
require 'digest'
require 'fileutils'

def generate_hashed_filename(file_path)
  # Read the file's contents
  content = File.read(file_path)

  # Generate a SHA-256 hash of the content
  digest = Digest::SHA256.hexdigest(content)

  # Construct the new filename with the hash
  # appended
  dir = File.dirname(file_path)
  ext = File.extname(file_path)
  basename = File.basename(file_path, ext)
  hashed_filename = "#{basename}-#{digest[0...8]}#{ext}"

  # Return the new path
  File.join(dir, hashed_filename)
end
```



You could create a function that will create a hash based on if the contents of the file has changed.

So if the content hash was content hash of
d41d8cd98f00b204e9800998ecf8427e

This function would output a file called:

- style-d41d8cd9.css

Cache busting using Cache Digest Hash is a common strategy for static websites or assets.

Javascript bundlers like webpacker can automate this process of creating digest hashes.

CloudFront – Default Root Object

CloudFront can be configured to return a specific object as the **default root object**.

The Default, default root object is **index.html**

`https://d111111abcdef8.cloudfront.net/`

Would return...

`https://d111111abcdef8.cloudfront.net/index.html`

If you changed the default root object to **hello/world.jpg**

`https://d111111abcdef8.cloudfront.net/`

Would return...

`https://d111111abcdef8.cloudfront.net/hello/world.jpg`

If a user requests a subdirectory of your CloudFront distribution it will not return the default root object

CloudFront – Requiring HTTPS

You can configure CloudFront to **require that viewers use HTTPS** so that connections are encrypted when CloudFront communicates with viewers.

- **Requiring HTTPS between viewers and CloudFront**
 - When using CloudFront domain name:
 - change the Viewer Protocol Policy setting for one or more cache behaviors to require HTTPS communication.
 - **When using CloudFront with a custom domain name:**
 - Create or Import an SSL/TLS Certificate via AWS Certification Manager (AWS)
- **Requiring HTTPS to a custom origin**
 - Change the Origin Protocol Policy
 - Install an SSL/TLS certificate on your Origin Server
- **Requiring HTTPS to an Amazon S3 origin**
 - When an Origin points to a bucket change the Viewer Protocol Policy to require HTTPS
 - When an Origin points to a bucket website endpoint than HTTPS can't be required

To help thwart SSL renegotiation-type attacks, CloudFront does not support renegotiation for viewer and origin requests.

CloudFront – Signed URLs

CloudFront signed URLs **allows users access to your private content** via temporarily generated URL

A CloudFront Signed URL can use either **canned** or **custom** policy

```
{
  "Statement": [
    {
      "Resource": "base URL or stream name",
      "Condition": {
        "DateLessThan": {
          "AWS:EpochTime": 1710881151
        }
      }
    }
  ]
}
```

A **canned policy** just contains the path the URL and when it should expire

```
{
  "Statement": [{
    "Resource": "(Optional) URL of the file>",
    "Condition": {
      "DateLessThan": {
        "AWS:EpochTime": 1710881151
      },
      "DateGreaterThan": {
        "AWS:EpochTime": 1710882131
      },
      "IpAddress": {
        "AWS:SourceIp": "<Optional: IP address>"
      }
    }
  }]
}
```

A **custom policy** has more restrictions that can be applied to the signed url

A Signed URL policy document must be signed using **a CloudFront key pair**
The key pair can be generated via the CloudFront console.

CloudFront – Signed URLs

A CloudFront Signed URL has to **be generated programmatically** since you need to sign the policy document using the CloudFront key pair.



```
def create_cloudfront_signed_url(distribution_domain, s3_key, key_pair_id, private_key_path, expires_in):
    """Generate a CloudFront signed URL."""
    # Construct the URL to the S3 object
    url = f"https://{distribution_domain}/{s3_key}"
    # Set expiration time
    expires = int(time.time()) + expires_in
    # Create the canned policy
    canned_policy = f'{{"Statement":[{{"Resource":"{url}","Condition":{{"DateLessThan":{{"AWS:EpochTime":{expires}}}}}}}}}'
    # Load the CloudFront private key from file
    with open(private_key_path, 'r') as file:
        private_key_data = file.read()
    private_key = rsa.PrivateKey.load_pkcs1(private_key_data.encode('utf-8'))
    # Sign the canned policy
    signature = sign_with_rsa(private_key, canned_policy)
    # URL-safe base64 encode the signature
    encoded_signature = url_safe_base64_encode(signature)
    # Construct the signed URL
    signed_url = f"{url}?Expires={expires}&Signature={quote_plus(encoded_signature)}&Key-Pair-Id={key_pair_id}"
    return signed_url
```

CloudFront – Signed Cookies

CloudFront Signed Cookies allows users access to your private content via cookies.

- Signed Cookies use the same syntax for canned or custom policy documents.
- Signed Cookies policy documents must be signed using an CloudFront key pair just like Signed Urls
- Signed Cookies makes sense when you need to let users access multiple files
 - Imagine a CloudFront distrubtion that serves videos and these videos should only be accessible to paid users.

You need to create three cookies that will be served by our web-application:

- CloudFront-Policy
- CloudFront-Signature
- CloudFront-Key-Pair-Id.

CloudFront – Signed Cookies

Creating a **signed cookie**.

```
def create_signed_cookie(resource_url, key_pair_id, private_key_path, expires_in_seconds):
    # Define the policy statement
    policy = {
        "Statement": [{
            "Resource": resource_url,
            "Condition": {
                "DateLessThan": {
                    "AWS:EpochTime": int(datetime.now(tz=timezone.utc).timestamp()) + expires_in_seconds
                }
            }
        }]
    }
    policy_string = json.dumps(policy).encode('utf-8')
    # Load the CloudFront private key
    with open(private_key_path, 'r') as file:
        private_key_data = file.read()
    private_key = rsa.PrivateKey.load_pkcs1(private_key_data.encode('utf-8'))
    # Sign the policy with the RSA private key
    signature = rsa.sign(policy_string, private_key, 'SHA-1')
    # URL-safe Base64 encode the policy and signature
    encoded_policy = base64.urlsafe_b64encode(policy_string).decode('utf-8').replace('=', '')
    encoded_signature = base64.urlsafe_b64encode(signature).decode('utf-8').replace('=', '')

    # Return the cookies
    return {
        'CloudFront-Policy': encoded_policy,
        'CloudFront-Signature': encoded_signature,
        'CloudFront-Key-Pair-Id': key_pair_id
    }
```

Notice the **following cookies that will be set**

CloudFront – Custom Error Pages

CloudFront distribution can be configured to serve up custom error pages.

- You can change the path to a custom error page for your origin
- You can change the response code returned

Custom error pages is useful when you want to serve a **Single Page App (SPA)** via Amazon S3 since you can route all 404s to the index.html

```
"CustomErrorResponses": {
  "Quantity": 2,
  "Items": [
    {
      "ErrorCode": 404,
      "ResponsePagePath": "/custom-404.html",
      "ResponseCode": "404",
      "ErrorCachingMinTTL": 300
    },
    {
      "ErrorCode": 500,
      "ResponsePagePath": "/custom-500.html",
      "ResponseCode": "500",
      "ErrorCachingMinTTL": 300
    }
  ]
},
```

CloudFront – Compressed Files

CloudFront can automatically compress and serve files for your origin.

Using a Cache Policy you can compress files using:

- Gzip
- Brotli